Final Project Report

Smart Retail Analytics Hub

ISM4545.001

Ana Luiza Jacob, Anjali Krishna, Harram Zahid, Preciosa Ngoy Kalumba,

11/10/2025

Executive Summary

This project presents the design and implementation of a hybrid data management system that integrates Online Transaction Processing (OLTP), Online Analytical Processing (OLAP), and JSON Relational Duality Views within a unified Oracle 23ai environment. The primary objective of the project is to model a realistic retail business scenario in which transactional data can be efficiently captured, maintained, and analyzed to support data-driven decision-making and future AI applications.

The OLTP component serves as the foundation of the system, encompassing core entities such as Customers, Orders, Products, Stores, and Inventory. The schema was normalized to Third Normal Form (3NF) to ensure data consistency, eliminate redundancy, and enforce referential integrity through foreign key constraints and triggers. Transactional operations, such as inventory adjustments and order total maintenance, were automated through delta-based triggers, ensuring real-time data accuracy across all interrelated tables.

Building upon this transactional foundation, the OLAP layer restructures the operational data into a Star Schema, designed to enable efficient aggregation, trend analysis, and multidimensional reporting. The central Fact_Sales table connects with several dimension tables—Dim_Customer, Dim_Product, Dim_Store, and Dim_Time—to support analytical exploration across customer demographics, product categories, geographical regions, and temporal hierarchies. This design enables complex analytical queries and facilitates the development of business intelligence dashboards for strategic insight.

The system was implemented using Oracle 23ai, chosen for its advanced support of hybrid workloads and seamless integration between relational and semi-structured data through JSON duality views. SQL and PL/SQL were used for schema creation, data transformation, and analytical query development.

Key deliverables of the project include:

- A fully normalized OLTP schema that supports core business transactions.
- An OLAP Star Schema optimized for analytical queries and data cube operations.
- Implementation of JSON Relational Duality Views for flexible data access.
- Demonstration of analytical queries that reveal actionable business insights.

Introduction

In the era of data-driven business operations, the retail industry increasingly depends on integrated data systems to extract meaningful insights from both transactional and analytical sources. Retail analytics has evolved from traditional sales reporting to advanced predictive systems that leverage hybrid data architectures and artificial intelligence (AI). This project, titled "Smart Retail Analytics Hub," addresses the need for a unified data environment capable of supporting real-time transactions, historical trend analysis, and AI-ready insights for an enterprise-scale retail business.

The primary goal of this project is to design and implement a hybrid data platform that bridges Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP) systems within a single Oracle 23ai environment. The system enables a retail organization to simultaneously

manage day-to-day sales transactions while also generating multidimensional analytics for business intelligence. Additionally, the project incorporates JSON Relational Duality Views, which facilitate the representation of relational data in a document-style format to support customer-facing applications and AI-driven use cases.

The scope of the project encompasses three major components. First, a relational OLTP schema was developed to capture and maintain core transactional data related to customers, products, stores, orders, and inventory. Second, the OLTP schema was transformed into a Star Schema OLAP model, designed to support efficient aggregation, trend analysis, and visualization of key performance indicators such as total sales revenue, units sold, and regional performance. Finally, a JSON-based dual data view layer was implemented to provide flexible, AI-ready access to nested customer and order data structures without compromising data integrity.

By combining these approaches, the project demonstrates how traditional relational modeling, dimensional analytics, and semi-structured data technologies can be integrated into a cohesive ecosystem. The outcome is a scalable, intelligent retail data platform capable of supporting both operational and strategic decision-making, illustrating the practical application of hybrid data management concepts in contemporary business environments.

Justification of Design Decisions

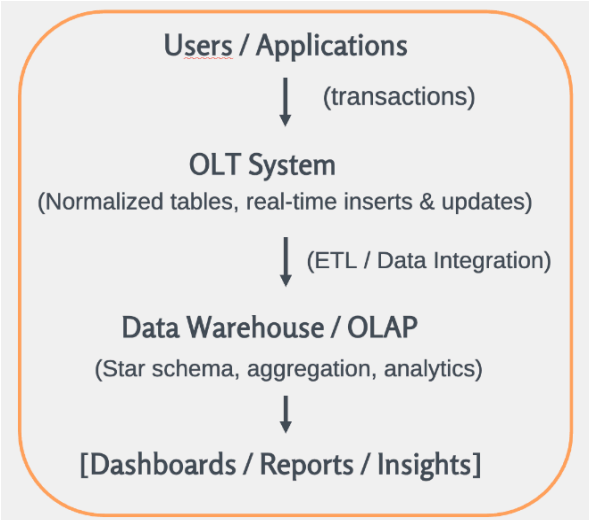
The design of the Smart Retail Analytics Hub was guided by the goal of integrating operational efficiency, analytical depth, and flexibility for future scalability. The use of a normalized OLTP schema (3NF) ensures data integrity and supports real-time transaction processing, while reducing redundancy and maintaining consistency across customers, products, and orders. This foundation was extended with an OLAP star schema, enabling fast aggregation and multidimensional analysis through clearly defined fact and dimension tables. The trade-off of increased storage in OLAP is justified by the significant gains in query performance and usability for business intelligence.

To handle unstructured and evolving data, the system incorporates a JSON Document Store, which accommodates flexible customer profiles and behavioral data that cannot be captured within traditional relational models. This enables seamless integration with AI-driven applications and personalization systems. Complementing this, JSON Relational Duality Views bridge the relational and document paradigms—allowing unified access and updates to both structured and semi-structured data without duplication. Additionally, features such as inventory triggers, snapshot pricing, and targeted indexes ensure operational responsiveness and data accuracy at scale, even with large transaction volumes.

Overall, each design decision balances performance, scalability, and adaptability, reflecting a modern hybrid data architecture. By combining normalized relational design with analytical warehousing and JSON-based flexibility, the system supports both real-time operations and long-term strategic intelligence—positioning it as a robust, future-ready data solution for retail analytics.

1. System Architecture and Design

- High-level system architecture diagram



- Description of components:
    - Relational schema (OLTP) - Online Transaction Processing

Basically, the OLTP system serves as the operational backbone of the Smart Retail Analytics Hub, handling all day-to-day transactional activities in real time. It stores and manages detailed data related to customers, products, orders, and inventory, ensuring the integrity and accuracy of ongoing business operations.

Through SQL-based operations such as INSERT, UPDATE, and DELETE, the system continuously records events like new customer purchases, payment confirmations, inventory adjustments, and shipment updates. Because these transactions occur instantly, the OLTP database provides a real-time view of business performance, supporting front-line operations such as sales processing, order fulfillment, and customer service.

To maintain high performance and data consistency, the schema is normalized (up to Third Normal Form)—reducing redundancy and enabling fast, reliable transaction execution. Periodically, this transactional data is extracted and transferred to the OLAP layer, where it can be aggregated and analyzed for long-term insights and decision-making.

Overall, the OLTP system ensures that operational data remains current, accurate, and actionable, forming the foundation for both business execution and analytical intelligence in the broader data ecosystem.

1.1 Data Dictionary

Tables

CUSTOMERS

- **PK:** customer_id NUMBER IDENTITY
- **Columns:**
  first_name VARCHAR2(50) NOT NULL, last_name VARCHAR2(50) NOT NULL,
  email VARCHAR2(120) NOT NULL UNIQUE, phone VARCHAR2(25),
  created_at TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
  status VARCHAR2(20) DEFAULT 'ACTIVE' CHECK (IN 'ACTIVE','INACTIVE')
- **Notes:** email uniqueness prevents duplicates; status controls eligibility.

STORES

- **PK:** store_id
- **Columns:**
  store_name VARCHAR2(100) NOT NULL, region VARCHAR2(50),
  city VARCHAR2(60), state_province VARCHAR2(60),
  created_at TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL
- **Notes:** Region/location feed OLAP dimensions.

PRODUCTS

- **PK:** product_id
- **Columns:**
  sku VARCHAR2(40) NOT NULL UNIQUE, product_name VARCHAR2(120) NOT NULL,
  category VARCHAR2(80), unit_price NUMBER(10,2) CHECK (>=0) NOT NULL,
  active_flag CHAR(1) DEFAULT 'Y' CHECK (IN 'Y','N'),
  created_at TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL
- **Notes:** sku is a natural key; unit_price is current price (not historical)

INVENTORY

- **PK:** (store_id, product_id)

- **FKs:** store_id → STORES.store_id, product_id → PRODUCTS.product_id

- **Columns:**
  on_hand_qty NUMBER(12,0) DEFAULT 0 CHECK (>=0),
  safety_stock_qty NUMBER(12,0) DEFAULT 0 CHECK (>=0),
  reorder_point_qty NUMBER(12,0) DEFAULT 0 CHECK (>=0),
  last_updated TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL

- **Business rules:** No negative stock; triggers adjust stock on sales.

ORDERS

- **PK:** order_id NUMBER IDENTITY

- **FKs:** customer_id → CUSTOMERS, store_id → STORES

- **Columns:**
  order_datetime TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,

order_status VARCHAR2(20) DEFAULT 'PLACED' CHECK (IN 'PLACED','PAID','CANCELLED','FULFILLED'),
payment_method VARCHAR2(20) CHECK (IN 'CREDIT','DEBIT','CASH','GIFT','OTHER'),
order_total_amt NUMBER(12,2) DEFAULT 0 NOT NULL CHECK (>=0)

- **Purpose:** Order header: who, where, when, how much.
- **Notes:** order_total_amt maintained by delta triggers from line items.

ORDER_ITEMS

- **PK:** order_item_id NUMBER IDENTITY
- **FKs:** order_id → ORDERS, product_id → PRODUCTS
- **Columns:**
  quantity NUMBER(12,0) CHECK (>0) NOT NULL,
  unit_price_at_sale NUMBER(10,2) CHECK (>=0) NOT NULL,
  line_amount VIRTUAL AS (quantity * unit_price_at_sale)
- **Purpose:** Order line details.
- **Notes:** unit_price_at_sale is a **snapshot** to preserve history.

Cardinalities:

- Customer has many Orders
- Store has many Orders
- Order has many Order_Items
- Product appears in many Order_Items

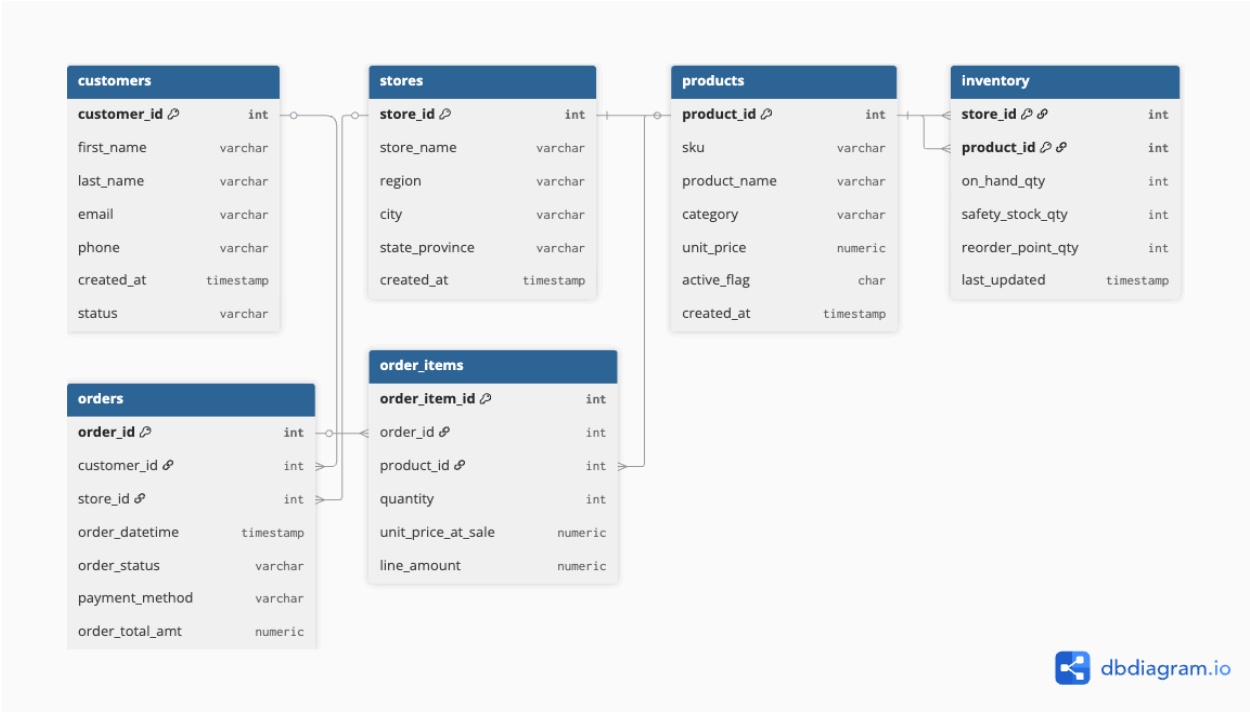
1.2 Relationships and normalization (3NF).

The schema is normalized to Third Normal Form:

- Each table stores facts about a single entity (no repeating groups for ex: (Order vs. Order_Items)
- All non-key attributes depend **only** on the key (e.g., ORDERS.payment_method depends on order_id, not on customer_id).
- No transitive dependencies (e.g., product attributes aren't stored on ORDER_ITEMS; instead product_id links to PRODUCTS).

- Keys identify rows; non-keys depend only on the key (e.g., product_price lives in Products, not Order_Items; we copy **unit_price_at_sale** to Order_Items deliberately as a historical snapshot(this is allowed and common in OLTP).

ERD (Entity-Relationship Diagram).



  - OLAP star schema and cubes

JSON Document Store

The JSON Document Store layer introduces flexibility for unstructured and semi-structured data that cannot be easily represented in fixed relational columns.
It is particularly useful for capturing customer profiles, behavioral data, and application-layer metadata, where fields may evolve over time or differ across users.
This layer is implemented using Oracle's native JSON data type, allowing direct querying and indexing of JSON fields using SQL.
It enhances the retail system's adaptability to new business needs such as customer personalization, AI-driven insights, and integration with front-end mobile applications.

JSON Relational Duality Views

The JSON Relational Duality Views act as a bridge between structured relational data and hierarchical JSON representations. They enable relational tables to be accessed and manipulated as document-style JSON views, without duplicating or desynchronizing data.
This means that developers and data scientists can insert, update, or delete nested JSON objects, while Oracle automatically propagates those changes to the appropriate relational tables.

In the Smart Retail Analytics Hub, these views simplify data access for modern applications and API services, offering a unified, hybrid model that supports both SQL-based analytics and JSON-based operational logic.

Here is a table with the Justification of design decisions

Decision	Purpose	Trade-off
Normalized OLTP (3NF)	Ensures data accuracy and supports real-time transactions.	More joins but higher consistency.
Snapshot Pricing	Keeps historical sales records accurate for reporting.	Minor duplication of price data.
Inventory Triggers	Automatically updates stock and order totals in real time.	Added system complexity.
Targeted Indexes	Improves query and join performance.	Slightly slower write speed.
Star Schema (OLAP)	Enables efficient and fast analytical queries.	Higher storage usage.
Time Dimension	Simplifies date-based reporting and grouping.	Requires periodic refresh.
JSON Tables	Allows storage of flexible, semi-structured data.	Less strict validation.
Duality Views	Combines relational and document data access.	More complex setup.
Surrogate Keys	Simplifies joins and ensures key stability.	Adds extra columns.
ETL/ELT Pipeline	Transfers clean data from OLTP to OLAP efficiently.	Data slightly delayed (near real-time).
Governance Rules	Ensures data quality and consistency.	More initial design effort.
Oracle 23ai	Supports hybrid SQL and JSON processing in one platform.	Reduced portability.

2. Implementation

2.1 Relational Database

- ER diagram
- Normalized schema description and SQL table (metadata) definitions
- Sample transactional queries

2.2 OLAP Star Schema

Objective

The goal of this stage is to transform the normalized **OLTP retail database** into a **dimensional (OLAP) model** suitable for advanced analytics and business intelligence. While the OLTP system handles day-to-day transactions, the OLAP model aggregates data for trend analysis, performance tracking, and strategic decision-making.

2.1 Star Schema Design

The proposed **Star Schema** centers around a single fact table, FACT_SALES, surrounded by four dimension tables:

Table	Type	Description
FACT_SALES	Fact	Core table storing measurable sales transactions
DIM_CUSTOMER	Dimension	Descriptive attributes about customers
DIM_PRODUCT	Dimension	Product details and categories
DIM_STORE	Dimension	Store locations and regions
DIM_TIME	Dimension	Calendar-based breakdowns for time analysis

Star Schema Design Visual:

```
      DIM_CUSTOMER
                |
                |
  DIM_PRODUCT — FACT_SALES — DIM_STORE
```

|

|

DIM_TIME

Dimension Table Creation

Code for Dimensions:

```
-- Customer Dimension

CREATE TABLE dim_customer AS

SELECT

    c.customer_id AS customer_key,

    c.first_name,

    c.last_name,

    c.email,

    c.status,

    c.created_at

FROM customers c;


-- Product Dimension

CREATE TABLE dim_product AS

SELECT

    p.product_id AS product_key,

    p.sku,

    p.product_name,

    p.category,

    p.unit_price,

    p.active_flag
```

```
FROM products p;

-- Store Dimension
CREATE TABLE dim_store AS
SELECT
    s.store_id AS store_key,
    s.store_name,
    s.region,
    s.city,
    s.state_province
FROM stores s;

-- Time Dimension
CREATE TABLE dim_time AS
SELECT DISTINCT
    TRUNC(o.order_datetime) AS date_key,
    EXTRACT(YEAR  FROM o.order_datetime) AS year,
    EXTRACT(MONTH FROM o.order_datetime) AS month,
    TO_CHAR(o.order_datetime, 'Month')  AS month_name,
    TO_CHAR(o.order_datetime, 'Q')      AS quarter
FROM orders o;
```

Fact Table Creation

Code for fact tables:

```
CREATE TABLE fact_sales AS
SELECT
```

```
    oi.order_item_id      AS sales_id,
    o.order_id,
    o.customer_id         AS customer_key,
    o.store_id            AS store_key,
    oi.product_id         AS product_key,
    TRUNC(o.order_datetime) AS date_key,
    oi.quantity,
    oi.unit_price_at_sale,
    (oi.quantity * oi.unit_price_at_sale) AS line_total,
    o.payment_method,
    o.order_status
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id;
```

Adding the relationship and Integrity constraints:

```
-- Dimension Primary Keys
ALTER TABLE dim_customer ADD CONSTRAINT pk_dim_customer PRIMARY KEY (customer_key);
ALTER TABLE dim_product  ADD CONSTRAINT pk_dim_product  PRIMARY KEY (product_key);
ALTER TABLE dim_store    ADD CONSTRAINT pk_dim_store    PRIMARY KEY (store_key);
ALTER TABLE dim_time     ADD CONSTRAINT pk_dim_time     PRIMARY KEY (date_key);
```

-- Fact Foreign Keys

Measures and Hierarchies

Measures (quantitative metrics):

- `quantity`
- `line_total` (revenue)
  Hierarchies (aggregation paths):
- **Time:** `Year → Quarter → Month → Date`
- **Product:** `Category → Product Name`
- **Store:** `Region → City → Store`

Example Analytical Queries

Below are SQL examples demonstrating how this star schema supports common BI use cases.

Total Revenue by Product Category

```
SELECT dp.category,

       SUM(fs.line_total) AS total_revenue

FROM fact_sales fs

JOIN dim_product dp ON fs.product_key = dp.product_key

GROUP BY dp.category

ORDER BY total_revenue DESC;
```

Monthly Revenue by Region

```
SELECT dt.year, dt.month_name, ds.region,

       SUM(fs.line_total) AS total_sales

FROM fact_sales fs

JOIN dim_time  dt ON fs.date_key = dt.date_key

JOIN dim_store ds ON fs.store_key = ds.store_key

GROUP BY dt.year, dt.month_name, ds.region

ORDER BY dt.year, dt.month, ds.region;
```

Top-5 Products by Sales Revenue

```
SELECT dp.product_name,
       SUM(fs.line_total) AS total_revenue
FROM fact_sales fs
JOIN dim_product dp ON fs.product_key = dp.product_key
GROUP BY dp.product_name
ORDER BY total_revenue DESC
FETCH FIRST 5 ROWS ONLY;
```

2.3 JSON Document Store

While the OLTP and OLAP components of the Smart Retail Analytics Hub ensure structured and historical data management, modern retail systems increasingly require the ability to handle unstructured and rapidly changing data generated by customer interactions and digital channels. To address this need, the project extends the relational foundation with JSON-based data management features within Oracle 23ai. These components, the JSON Document Store and JSON Relational Duality Views, enable the system to store, query, and modify complex hierarchical data in native JSON format while maintaining the transactional integrity of the underlying relational schema. This hybrid approach enhances the adaptability of the retail platform by combining traditional database design with the flexibility required for AI integration and real-time analytics.

Structure of JSON Documents

The JSON Document Store was implemented to handle customer-related and behavior-driven data that are inherently flexible and semi-structured.
Three dedicated tables were created:

CUSTOMER_PROFILE_JSON (CUSTOMER_ID, PROFILE JSON)

Stores detailed customer profiles, including personal details, contact preferences, loyalty information, and marketing opt-in choices.

Each profile is represented as a single JSON document containing nested objects and arrays.

CUSTOMER_PREF_EVENT_JSON (PREF_EVENT_ID, CUSTOMER_ID, EVENT_DOC JSON)

Records behavioral events such as added_to_cart, viewed_product, or clicked_promotion, along with contextual metadata like device type and campaign source.

This table allows analysis of customer engagement patterns over time.

ORDER_JSON_CACHE (ORDER_ID, ORDER_DOC JSON)

Maintains denormalized order summaries for quick access by mobile apps or dashboards.

Each JSON document represents an entire order, including customer details, store data, and an array of purchased items.

These JSON tables support nested data structures that capture the complexity of real-world entities.
A representative example of a customer profile document is shown below:

```
{
  "name": {"first": "Alex", "last": "Rivera"},
  "contacts": {"email": "alex.r@example.com", "phone": "555-1010"},
  "preferences": {
    "sizes": ["M"],
    "styles": ["minimal", "athleisure"],
    "brands": ["Acme", "Nimbus"],
    "channels": ["email", "sms"]
  },
  "marketingOptIn": true,
  "loyalty": {"tier": "GOLD", "points": 12450}
};
```

This flexible structure eliminates the need to alter database schemas when new attributes are introduced, supporting long-term scalability and agility.

```
SQL> CREATE TABLE ORDER_JSON_CACHE (
       ORDER_ID  NUMBER PRIMARY KEY
         REFERENCES ORDERS(ORDER_ID) ON DELETE CASCADE,
       ORDER_DOC JSON...
Show more...



Table ORDER_JSON_CACHE created.
```

Figure 5.3-A. JSON tables created successfully in Oracle 23ai.

```sql
-- Verify inserts
SELECT c.CUSTOMER_ID,
       JSON_VALUE(p.PROFILE,'$.preferences.channels[0]') AS primary_channel,
       JSON_VALUE(p.PROFILE,'$.loyalty.tier')            AS loyalty_tier
FROM CUSTOMERS c
JOIN CUSTOMER_PROFILE_JSON p ON p.CUSTOMER_ID = c.CUSTOMER_ID
WHERE c.CUSTOMER_ID = 1;

SELECT PREF_EVENT_ID,
       CUSTOMER_ID,
       JSON_VALUE(EVENT_DOC,'$.event') AS event_type
FROM CUSTOMER_PREF_EVENT_JSON
WHERE CUSTOMER_ID = 1
ORDER BY PREF_EVENT_ID;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.012 seconds

	CUSTOMER_ID	PRIMARY_CHANNEI	LOYALTY_TIER
1	1	email	GOLD

Figure 5.3-B. Example query retrieving nested JSON attributes.

```
SQL> CREATE INDEX IX_PREF_EVENT
       ON CUSTOMER_PREF_EVENT_JSON (
         JSON_VALUE(EVENT_DOC, '$.event' RETURNING VARCHAR2(40))
       )

Index IX_PREF_EVENT created.
```

```
SQL> CREATE INDEX IX_PROF_MARKETING
       ON CUSTOMER_PROFILE_JSON (
         JSON_VALUE(PROFILE, '$.marketingOptIn' RETURNING NUMBER)
       )

Index IX_PROF_MARKETING created.
```

Figure 5.3-C. JSON search indexes enhancing query performance.

Use Cases for Unstructured or Semi-Structured Big Data

The JSON Document Store enables the Smart Retail Analytics Hub to manage and process data types that extend beyond the capabilities of structured relational systems.

Key use cases include:

- **Dynamic Customer Profiling:**
  Marketing teams can record new behavioral or demographic attributes without redesigning database tables.

- **AI and Recommendation Systems:**
  JSON event data feeds machine learning models for next-best-offer recommendations and personalized promotions.

- **Omnichannel Integration:**
  JSON documents can store app-specific or campaign-specific information, supporting seamless synchronization with web, mobile, and third-party platforms.

- **Historical Behavior Tracking:**
  Events stored as JSON enable time-series analysis of how customers interact with products and promotions.

Through this approach, the system maintains the structure of operational data while gaining the flexibility required for innovation and AI integration.

Example JSON Documents and Queries

The JSON data can be queried and analyzed directly using SQL expressions.
Oracle 23ai allows navigation within nested JSON paths and filtering with JSON_EXISTS conditions.
Representative queries include:

```
-- Marketing-opted-in customers and their preferred channel
SELECT c.customer_id,
       JSON_VALUE(p.profile, '$.preferences.channels[0]') AS primary_channel
FROM customers c
JOIN customer_profile_json p ON p.customer_id = c.customer_id
WHERE JSON_EXISTS(p.profile, '$?(@.marketingOptIn == true)');

-- Identify customers who prefer brand "Acme" and size "M"
SELECT customer_id
FROM customer_profile_json
WHERE JSON_EXISTS(profile, '$?(@.preferences.brands[*]?(@ == "Acme"))')
  AND JSON_EXISTS(profile, '$?(@.preferences.sizes[*]?(@ == "M"))');

-- Determine most frequently added_to_cart SKUs
SELECT jt.sku, COUNT(*) AS adds
FROM customer_pref_event_json e,
     JSON_TABLE(e.event_doc, '$.items[*]'
       COLUMNS (sku VARCHAR2(40) PATH '$.sku')) jt
WHERE JSON_VALUE(e.event_doc, '$.event') = 'added_to_cart'
GROUP BY jt.sku
ORDER BY adds DESC;
```

These examples demonstrate Oracle's ability to handle JSON data natively while retaining the power of SQL-based querying and indexing.

2.4 JSON Relational Duality Views

Description and Purpose of Each View

Oracle 23ai introduces **JSON Relational Duality Views (JDVs)** as a mechanism for bridging structured relational data and flexible JSON documents within the same database.
A JDV allows a relational table (or set of joined tables) to be viewed and manipulated as a single JSON document while maintaining full transactional integrity.
This ensures that operations performed through the JSON interface are reflected immediately in the relational schema.

Two duality views were created for the Smart Retail Analytics Hub:

1. **ORDER_DV (Order Duality View):**
   Represents each order as a complete JSON document containing order metadata, the associated store and customer details, and an array of line items.
   This view simplifies data exchange with applications and allows developers to manage orders as single, self-contained JSON objects.

2. **CUSTOMER_DV (Customer Duality View):**
   Consolidates each customer's information, including relational attributes and the JSON-based profile document, into one unified view.
   It also embeds a summary of recent orders for quick reference.
   This view supports a "customer 360" perspective useful for CRM and analytics applications.

These views combine the reliability of relational data management with the flexibility of document-based modeling.

SQL Definition and Syntax of Views

The following statement defines the **ORDER_DV** view, which aggregates data from multiple base tables into a single nested JSON representation:

```
CREATE OR REPLACE JSON RELATIONAL DUALITY VIEW order_dv AS
SELECT JSON {
  'orderId'   : o.order_id,
  'placedAt'  : o.order_datetime,
  'status'    : o.order_status,
  'payment'   : o.payment_method,
  'totals'    : { 'amount' : o.order_total_amt },
  'store'     : {
      'storeId' : s.store_id,
      'name'    : s.store_name,
      'city'    : s.city,
      'state'   : s.state_province
  },
  'customer'  : {
      'customerId' : c.customer_id,
      'firstName'  : c.first_name,
      'lastName'   : c.last_name,
      'email'      : c.email
  },
```

```
  'items'     : [
      SELECT JSON {
        'lineNo'    : oi.order_item_id,
        'productId' : oi.product_id,
        'sku'       : p.sku,
        'name'      : p.product_name,
        'qty'       : oi.quantity,
        'unitPrice' : oi.unit_price_at_sale,
        'amount'    : oi.line_amount
      }
      FROM order_items oi
      JOIN products p ON p.product_id = oi.product_id
      WHERE oi.order_id = o.order_id
  ]
}
WITH KEYS (o.order_id, s.store_id, c.customer_id)
FROM orders o
JOIN stores s ON s.store_id = o.store_id
JOIN customers c ON c.customer_id = o.customer_id;
```

A similar syntax was used for **CUSTOMER_DV**, embedding the customer's profile JSON and up to 20 most recent orders.
Both definitions appear in the appendix of this report.

Read/Write Examples and Use Cases

The JSON Relational Duality Views fully support CRUD operations (**Create, Read, Update,**

and Delete) on JSON documents that are automatically synchronized with the underlying relational data.

- **Create Operation:**
  Inserting a new JSON document into `ORDER_DV` creates new entries in the `ORDERS` and `ORDER_ITEMS` tables simultaneously.
  This simplifies API-based order creation.

```
INSERT INTO order_dv VALUES ( JSON('{
  "placedAt": "2025-11-09T16:05:00",
  "status": "PAID",
  "payment": "CREDIT",
  "store": {"storeId": 1},
  "customer": {"customerId": 1},
  "items": [
    {"productId": 200, "qty": 2, "unitPrice": 19.99},
    {"productId": 300, "qty": 1, "unitPrice": 12.00}
  ]
}') );
```

- **Read Operation:**
  Applications can query `ORDER_DV` to retrieve each order as a single, formatted JSON document suitable for REST API responses.

- **Update Operation:**
  Modifications to nested JSON fields automatically update corresponding rows in relational tables.
  For example, changing an order's status or item quantity within the JSON view updates the relevant columns in `ORDERS` and `ORDER_ITEMS`.

```
UPDATE order_dv d
SET d.document = JSON_TRANSFORM(d.document,
  SET '$.status' = 'CANCELLED')
WHERE JSON_VALUE(d.document, '$.orderId') = 2;
```

- **Delete Operation:**
  Deleting a document from the JSON view cascades deletions to all dependent relational records, maintaining referential integrity.

```
DELETE FROM order_dv d
WHERE JSON_VALUE(d.document, '$.orderId') = 2;
```

These operations demonstrate Oracle 23ai's capability to **synchronize structured and unstructured data seamlessly**, providing a single, hybrid representation that supports both SQL and JSON workloads.

Overall, the implementation of the JSON Document Store and JSON Relational Duality Views significantly enhances the Smart Retail Analytics Hub's flexibility and scalability. The document store provides a foundation for managing semi-structured and evolving datasets, while duality views unify relational and JSON-based access under one consistent model.

This hybrid design future-proofs the system by enabling integration with AI-driven personalization engines, real-time dashboards, and modern microservice-based retail applications, all without compromising transactional reliability or analytical depth.

3. Challenges and Solutions

4.1 Overview

This section demonstrates how the Smart Retail Analytics Hub converts transactional and analytical data into decision-ready insights. The BI portion surfaces key operational KPIs for replenishment, product performance, regional sales, and time trends. The AI portion prepares model-ready datasets for product recommendations and (optionally) enriches analysis with JSON behavioral data and JSON Relational Duality Views. Evidence for each use case is provided in the Appendix (Figures A1–A9), consistent with the project template.

4.2 Business Intelligence (BI) Use Cases

4.2.1 Real-Time Inventory "Low-Stock" Alerts

Purpose: identify SKUs at or below reorder thresholds across stores for proactive replenishment.

```sql
SELECT s.store_name,
       p.product_name,
       i.on_hand_qty,
       i.reorder_point_qty
FROM   inventory i
JOIN   products  p ON p.product_id = i.product_id
JOIN   stores    s ON s.store_id   = i.store_id
WHERE  i.on_hand_qty <= i.reorder_point_qty
ORDER  BY s.store_name, p.product_name;
```

STORE_NAME	PRODUCT_NAME	ON_HAND_QTY	REORDER_POINT_QTY
Airport Kiosk	Ergonomic Chair	15	20
Downtown Flagship	27in 4K Monitor	19	20
Downtown Flagship	Ergonomic Chair	14	20
Northside Mall	27in 4K Monitor	19	20
Northside Mall	Ergonomic Chair	15	20

Screenshot: A1 — rows flagged for replenishment.

Displays items where on-hand quantity is less than or equal to the reorder point, helping management detect products that require immediate replenishment.

4.2.2 Top-Performing Products (Revenue & Units)

Purpose: surface the Top-10 products by revenue for pricing and promotion decisions.

```
SELECT p.category,
       p.product_name,
       SUM(oi.quantity)     AS units_sold,
       SUM(oi.line_amount) AS total_revenue
FROM   order_items oi
JOIN   products    p  ON p.product_id = oi.product_id
JOIN   orders      o  ON o.order_id   = oi.order_id
GROUP  BY p.category, p.product_name
ORDER  BY total_revenue DESC
FETCH FIRST 10 ROWS ONLY
```

	CATEGORY	PRODUCT_NAME	UNITS_SOLD	TOTAL_REVENUE
1	Displays	27in 4K Monitor	2	558
2	Storage	Portable SSD 1TB	3	357
3	Accessories	Mechanical Keyboarc	3	267
4	Furniture	Ergonomic Chair	1	219
5	Accessories	Wireless Charger	5	145
6	Accessories	Wireless Mouse	5	124.95
7	Accessories	Webcam 1080p	3	119.97

Screenshot: A2 — top-10 table (works great for a bar chart).

Shows the highest-revenue products based on aggregated sales totals. This helps identify best-selling SKUs and supports pricing and marketing decisions.

4.2.3 Regional Sales & Active Customers

Purpose: track total sales and distinct active customers by region to guide allocation and campaigns.

```
SELECT s.region,
       COUNT(DISTINCT o.customer_id) AS active_customers,
```

```
       SUM(o.order_total_amt)        AS total_sales
FROM   orders o
JOIN   stores s ON s.store_id = o.store_id
GROUP  BY s.region
ORDER  BY total_sales DESC
```

	REGION	ACTIVE_CUSTOMER	TOTAL_SALES
1	East	10	2057.92

Screenshot: A3 — KPIs by region.

Aggregates total sales and number of active customers by region. It supports strategic planning and regional performance monitoring.

4.2.4 Daily Sales Trend

Purpose: monitor revenue trajectory and seasonality at a daily grain.

```
SELECT TRUNC(o.order_datetime) AS order_date,
       SUM(o.order_total_amt)  AS daily_sales
FROM   orders o
GROUP  BY TRUNC(o.order_datetime)
ORDER  BY order_date;
```

	ORDER_DATE	DAILY_SALES
1	11/10/2025, 12:00:00 AM	2057.92

Screenshot: A4 — daily totals table (good for a line chart).

Plots daily revenue totals to show demand patterns over time. Useful for detecting sales seasonality and operational peaks.

4.3 Artificial Intelligence (AI) Use Cases

4.3.1 Product Recommendations (Frequently Bought Together)

Foundation: co-occurrence analysis across order line items to generate “people also buy” pairs (dataset supports collaborative filtering models later). This satisfies the brief’s AI-driven

recommendations via SQL/duality view outputs.

Figure A5 – Frequently Bought Together Pairs (AI Use Case 1)

```
SELECT p1.product_name AS base_product,

       p2.product_name AS recommended_product,

       COUNT(*)        AS cooccurrence_count

FROM   order_items oi1

JOIN   order_items oi2

       ON oi1.order_id = oi2.order_id

      AND oi1.product_id <> oi2.product_id

JOIN   products p1 ON p1.product_id = oi1.product_id

JOIN   products p2 ON p2.product_id = oi2.product_id

GROUP  BY p1.product_name, p2.product_name

HAVING COUNT(*) >= 2

ORDER  BY cooccurrence_count DESC, base_product, recommended_product
```

	BASE_PRODUCT	RECOMMENDED_PR	COOCCURRENCE_C
1	27in 4K Monitor	Mechanical Keyboarc	2
2	Mechanical Keyboarc	27in 4K Monitor	2

Identifies co-purchased product pairs from order line items. This supports product-recommendation features and cross-selling analysis.

4.3.2 JSON-Driven Behavioral Signals

Oracle 23ai JSON tables and Duality Views (implemented by the team) allow AI features to incorporate profile and behavioral attributes without schema changes.

Example A — filter orders whose JSON “items” include the Accessories category:

```
SELECT o.order_id,

       JSON_VALUE(o.order_doc, '$.customer.email') AS customer_email,

       JSON_VALUE(o.order_doc, '$.total_amount')    AS order_total
```

```
FROM   order_json_cache o

WHERE  JSON_EXISTS(o.order_doc, '$.items[*]?(@.category == "Accessories")')

ORDER  BY o.order_id;
```

	ORDER_ID	CUSTOMER_EMAIL	ORDER_TOTAL
1	1	alex.r@example.com	457

Demonstrates how JSON queries extract order information where accessory items are included. This simulates how AI could segment transactions by category.

Example B — extract simple engagement events from behavior logs:

```
SELECT c.customer_id,

       JSON_VALUE(c.event_doc, '$.event')   AS event_name,

       JSON_VALUE(c.event_doc, '$.product') AS product_hint,

       JSON_VALUE(c.event_doc, '$.device')  AS device_type

FROM   customer_pref_event_json c

ORDER  BY c.customer_id
```

CUSTOMER_ID	EVENT_NAME	PRODUCT_HINT	DEVICE_TYPE

No items to display.

Shows JSON-stored customer events such as “viewed_product” or “added_to_cart.” These data enhance AI models for personalization and churn prediction.

4.3.3 Duality Views (“Customer 360” and Order Documents)

Duality Views expose nested JSON documents synchronized with relational bases (ORDER_DV, CUSTOMER_DV), simplifying downstream AI/BI consumption.

```
SELECT JSON_SERIALIZE(dv RETURNING CLOB PRETTY) AS customer_360_json

FROM   customer_dv

WHERE  JSON_VALUE(dv, '$.customer_id') = 1;  -- change 1 to an id that exists
```

	CUSTOMER_360_JSON
1	{ "customer_id" : 1, "first_name" : "Alex", "last_name" : "Rivera", "email" : "alex.r@example.com", "orders" : [{ "order_id" : 1, "order_datetime" : "2025-11-10T05:06:01", "order_total_

Presents a single JSON document summarizing each customer's profile and related orders, illustrating a 360-degree view concept for analytics.

```
SELECT JSON_SERIALIZE(dv RETURNING CLOB PRETTY) AS order_json

FROM   order_dv

WHERE  JSON_VALUE(dv, '$.order_id') = 1;   -- replace 1 with a real order id
```

	ORDER_JSON
1	{ "order_id" : 1, "customer_id" : 1, "store_id" : 1, "order_datetime" : "2025-11-10T05:06:01", "order_total" : 457, "items" : [{ "product_id" : 2, "quantity" : 2, "unit_price" : 89, "line_total" :

Displays a JSON representation of an individual order including items, quantities, and totals, enabling modern application or AI integration.

4. Conclusion

Summary of Achievements

The Smart Retail Analytics Hub project successfully demonstrates the practical integration of transactional, analytical, and unstructured data systems within a unified Oracle 23ai environment. The design captures the complete data lifecycle in a retail organization, from real-time transactions in the OLTP layer to multidimensional analysis in the OLAP schema and flexible semi-structured storage through the JSON Document Store. The addition of JSON Relational Duality Views further bridges structured and unstructured data, allowing real-time interactions through both SQL and document-based interfaces. Collectively, the system reflects a realistic enterprise data ecosystem that can process large volumes of data while remaining extensible for future integration with machine learning and AI-driven analytics. The outcome is a comprehensive architecture capable of supporting decision-making, personalization, and business intelligence at scale.

Integration of Course Skills

This project illustrates the application of core concepts and technical skills developed throughout the Big Data for AI course. The initial focus on understanding data's role in modern enterprises guided the normalization and relational design of the OLTP layer. Knowledge from the weeks covering **data warehousing, OLAP modeling, and distributed systems** informed the development of the star schema and analytical cubes used for business insights. The module on **Big Data Architecture and Infrastructure** supported the creation of efficient data pipelines and the management of data volume and velocity across transactional and analytical systems. Concepts from **Apache Spark, Hadoop, and NoSQL databases** were applied conceptually to understand how the JSON data structures could parallel scalable big data processing environments.

The lessons on **Big Data Governance, Ethics, and Security** reinforced the need to ensure data integrity and maintain privacy in customer-related JSON profiles. Additionally, the exploration of **cloud computing and data visualization tools** shaped the project's vision for extending analytical capabilities through dashboard integration and distributed access. Together, these elements demonstrate the practical use of theoretical knowledge gained across all course modules and highlight the student's ability to design, document, and analyze complex hybrid data systems in alignment with enterprise big data practices.

Potential Extensions or Future Enhancements

Future enhancements could extend the Smart Retail Analytics Hub toward a fully cloud-enabled, AI-driven system. Real-time data ingestion pipelines could be incorporated using tools such as Apache Kafka or Spark Streaming to process live transactions and behavioral events. The JSON Document Store can be expanded to include richer customer interactions such as online browsing patterns, product reviews, and social media feedback. These data sources could be utilized to train machine learning models for predictive analytics, product recommendations, and customer segmentation. Additionally, integrating Hadoop Distributed File System (HDFS) or Databricks-based data lakes would allow scalable data storage and advanced processing. Ethical considerations and governance mechanisms could also be strengthened to ensure responsible data usage and compliance with privacy standards. Finally, a cloud-based dashboard could be developed to visualize both OLAP and JSON-driven insights, offering management a unified interface for monitoring operational performance and strategic analytics.

5. Appendix

- Full SQL code and scripts

Part 1:

```
 1) CREATING TABLES (parents first, children later)
-- 1.1 CUSTOMERS  (parent)
CREATE TABLE customers (
 customer_id  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 first_name   VARCHAR2(50)  NOT NULL,
 last_name    VARCHAR2(50)  NOT NULL,
 email        VARCHAR2(120) NOT NULL UNIQUE,
 phone        VARCHAR2(25),
 created_at   TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
 status       VARCHAR2(20) DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','INACTIVE'))
);

-- 1.2 STORES  (parent)
CREATE TABLE stores (
 store_id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 store_name     VARCHAR2(100) NOT NULL,
 region         VARCHAR2(50),
 city           VARCHAR2(60),
 state_province VARCHAR2(60),
 created_at     TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL
);

-- 1.3 PRODUCTS  (parent)
CREATE TABLE products (
 product_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 sku          VARCHAR2(40)  NOT NULL UNIQUE,
 product_name VARCHAR2(120) NOT NULL,
 category     VARCHAR2(80),
 unit_price   NUMBER(10,2)  NOT NULL CHECK (unit_price >= 0),
 active_flag  CHAR(1) DEFAULT 'Y' CHECK (active_flag IN ('Y','N')),
 created_at   TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL
);

-- 1.4 ORDERS  (child of CUSTOMERS, STORES; parent of ORDER_ITEMS)
CREATE TABLE orders (
 order_id        NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 customer_id     NUMBER NOT NULL,
 store_id        NUMBER NOT NULL,
 order_datetime  TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
 order_status    VARCHAR2(20) DEFAULT 'PLACED'
```

```oracle
                    CHECK (order_status IN ('PLACED','PAID','CANCELLED','FULFILLED')),
 payment_method  VARCHAR2(20)
                    CHECK (payment_method IN ('CREDIT','DEBIT','CASH','GIFT','OTHER')),
 order_total_amt NUMBER(12,2) DEFAULT 0 NOT NULL CHECK (order_total_amt >= 0),
 CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES
customers(customer_id),
 CONSTRAINT fk_orders_store    FOREIGN KEY (store_id)    REFERENCES stores(store_id)
);

-- 1.5 INVENTORY  (child of STORES, PRODUCTS)
CREATE TABLE inventory (
 store_id          NUMBER NOT NULL,
 product_id        NUMBER NOT NULL,
 on_hand_qty       NUMBER(12,0) DEFAULT 0 CHECK (on_hand_qty >= 0),
 safety_stock_qty  NUMBER(12,0) DEFAULT 0 CHECK (safety_stock_qty >= 0),
 reorder_point_qty NUMBER(12,0) DEFAULT 0 CHECK (reorder_point_qty >= 0),
 last_updated      TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
 CONSTRAINT pk_inventory PRIMARY KEY (store_id, product_id),
 CONSTRAINT fk_inv_store   FOREIGN KEY (store_id)   REFERENCES stores(store_id),
 CONSTRAINT fk_inv_product FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- 1.6 ORDER_ITEMS  (child of ORDERS, PRODUCTS)
CREATE TABLE order_items (
 order_item_id      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 order_id           NUMBER NOT NULL,
 product_id         NUMBER NOT NULL,
 quantity           NUMBER(12,0) NOT NULL CHECK (quantity > 0),
 unit_price_at_sale NUMBER(10,2) NOT NULL CHECK (unit_price_at_sale >= 0),
 line_amount        AS (quantity * unit_price_at_sale) VIRTUAL,
 CONSTRAINT fk_oi_order   FOREIGN KEY (order_id)   REFERENCES orders(order_id),
 CONSTRAINT fk_oi_product FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- 2) INDEXES (create after table)
CREATE INDEX ix_orders_customer_dt ON orders (customer_id, order_datetime);
CREATE INDEX ix_oi_order           ON order_items (order_id);
CREATE INDEX ix_inventory_lowstock ON inventory (store_id, on_hand_qty);

-- 3) TRIGGERS (create AFTER tables)
--    A) Inventory integrity on ORDER_ITEMS changes
--    B) Order total maintained by delta (no mutating-table issue)

-- 3.A.1 After INSERT on ORDER_ITEMS: deduct inventory; create row if missing.
CREATE OR REPLACE TRIGGER trg_oi_inv_after_ins
AFTER INSERT ON order_items
```

```oracle_plsql
FOR EACH ROW
DECLARE
 v_store_id stores.store_id%TYPE;
 v_on_hand  inventory.on_hand_qty%TYPE;
BEGIN
 SELECT store_id INTO v_store_id FROM orders WHERE order_id = :NEW.order_id;

 UPDATE inventory
    SET on_hand_qty = on_hand_qty - :NEW.quantity,
        last_updated = SYSTIMESTAMP
  WHERE store_id = v_store_id
    AND product_id = :NEW.product_id
 RETURNING on_hand_qty INTO v_on_hand;

 IF SQL%ROWCOUNT = 0 THEN
   INSERT INTO inventory (store_id, product_id, on_hand_qty, last_updated)
   VALUES (v_store_id, :NEW.product_id, 0, SYSTIMESTAMP);

   UPDATE inventory
      SET on_hand_qty = on_hand_qty - :NEW.quantity,
          last_updated = SYSTIMESTAMP
    WHERE store_id = v_store_id
      AND product_id = :NEW.product_id
   RETURNING on_hand_qty INTO v_on_hand;
 END IF;

 IF v_on_hand < 0 THEN
   RAISE_APPLICATION_ERROR(-20001, 'Insufficient inventory for this item.');
 END IF;
END;
/
-- 3.A.2 After DELETE on ORDER_ITEMS: restore inventory.
CREATE OR REPLACE TRIGGER trg_oi_inv_after_del
AFTER DELETE ON order_items
FOR EACH ROW
DECLARE
 v_store_id stores.store_id%TYPE;
BEGIN
 SELECT store_id INTO v_store_id FROM orders WHERE order_id = :OLD.order_id;

 UPDATE inventory
    SET on_hand_qty = on_hand_qty + :OLD.quantity,
        last_updated = SYSTIMESTAMP
  WHERE store_id = v_store_id
    AND product_id = :OLD.product_id;
END;
```

```
/
-- 3.A.3 After UPDATE of QUANTITY on ORDER_ITEMS: lock row, check negative, apply
delta.
CREATE OR REPLACE TRIGGER trg_oi_inv_after_upd
AFTER UPDATE OF quantity ON order_items
FOR EACH ROW
DECLARE
 v_delta       NUMBER;
 v_store_id    NUMBER;
 v_new_on_hand NUMBER;
BEGIN
 v_delta := :NEW.quantity - :OLD.quantity;

 SELECT store_id INTO v_store_id
   FROM orders
  WHERE order_id = :NEW.order_id;

 SELECT on_hand_qty - v_delta
   INTO v_new_on_hand
   FROM inventory
  WHERE store_id = v_store_id
    AND product_id = :NEW.product_id
  FOR UPDATE;

 IF v_new_on_hand < 0 THEN
   RAISE_APPLICATION_ERROR(-20002, 'Update would result in negative inventory.');
 END IF;

 UPDATE inventory
    SET on_hand_qty = v_new_on_hand,
        last_updated = SYSTIMESTAMP
  WHERE store_id = v_store_id
    AND product_id = :NEW.product_id;
END;
/
-- 3.B) Order totals via DELTA triggers (avoid mutating-table readbacks)

-- 3.B.1 INSERT: add new line amount
CREATE OR REPLACE TRIGGER trg_oi_total_after_ins
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
 UPDATE orders
    SET order_total_amt = order_total_amt
                        + (:NEW.quantity * :NEW.unit_price_at_sale)
  WHERE order_id = :NEW.order_id;
```

```oracle_plsql
END;
/
-- 3.B.2 DELETE: subtract removed line amount
CREATE OR REPLACE TRIGGER trg_oi_total_after_del
AFTER DELETE ON order_items
FOR EACH ROW
BEGIN
 UPDATE orders
    SET order_total_amt = order_total_amt
                        - (:OLD.quantity * :OLD.unit_price_at_sale)
  WHERE order_id = :OLD.order_id;
END;
/
-- (run)

-- 3.B.3 UPDATE (quantity or price): adjust by delta
CREATE OR REPLACE TRIGGER trg_oi_total_after_upd
AFTER UPDATE OF quantity, unit_price_at_sale ON order_items
FOR EACH ROW
DECLARE
 v_old NUMBER := :OLD.quantity * :OLD.unit_price_at_sale;
 v_new NUMBER := :NEW.quantity * :NEW.unit_price_at_sale;
BEGIN
 UPDATE orders
    SET order_total_amt = order_total_amt + (v_new - v_old)
  WHERE order_id = :NEW.order_id;
END;
/
-- 4) SEED DATA (run AFTER triggers to exercise them)
-- 4.1 Seed customers
INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Alex','Rivera','alex.r@example.com','555-1010');
INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Blake','Chen','blake.c@example.com','555-2020');
INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Casey','Singh','casey.s@example.com','555-3030');

-- 4.2 Seed stores
INSERT INTO stores (store_name,region,city,state_province) VALUES ('Downtown
Flagship','East','Tampa','FL');
INSERT INTO stores (store_name,region,city,state_province) VALUES ('Northside
Mall','East','Tampa','FL');

-- 4.3 Seed products
INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-100','Wireless Mouse','Accessories',24.99);
```

```sql
INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-200','Mechanical Keyboard','Accessories',89.00);
INSERT INTO products (sku,product_name,category,unit_price) VALUES ('SKU-300','27in 4K
Monitor','Displays',279.00);
INSERT INTO products (sku,product_name,category,unit_price) VALUES ('SKU-400','USB-C
Cable 2m','Cables',12.50);

-- 4.4 Initialize inventory for all store/product pairs
INSERT INTO inventory (store_id, product_id, on_hand_qty, safety_stock_qty,
reorder_point_qty)
SELECT s.store_id, p.product_id,
      CASE p.sku WHEN 'SKU-300' THEN 20
                 WHEN 'SKU-200' THEN 50
                 WHEN 'SKU-100' THEN 100
                 ELSE 200 END,
      10, 20
FROM stores s CROSS JOIN products p;

-- 4.5 Example Order #1 (fires triggers)
INSERT INTO orders (customer_id, store_id, payment_method, order_status)
VALUES (1, 1, 'CREDIT', 'PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)
VALUES (1, (SELECT product_id FROM products WHERE sku='SKU-200'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-200'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)
VALUES (1, (SELECT product_id FROM products WHERE sku='SKU-300'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-300'));

UPDATE orders SET order_status='PAID' WHERE order_id = 1;
-- 5) QUICK VALIDATION (run AFTER seed)
-- 5.1 Confirm tables exist
SELECT table_name
FROM user_tables
WHERE table_name IN
('CUSTOMERS','STORES','PRODUCTS','INVENTORY','ORDERS','ORDER_ITEMS')
ORDER BY 1;
-- 5.2 Order total should be 457.00 for order 1
SELECT order_id, order_total_amt FROM orders WHERE order_id = 1;
-- 5.3 Inventory decreased for sold SKUs at store 1
SELECT i.store_id, p.sku, i.on_hand_qty
FROM inventory i
JOIN products p ON p.product_id = i.product_id
WHERE i.store_id = 1 AND p.sku IN ('SKU-200','SKU-300')
ORDER BY p.sku;
```

Adding more data in order to have relevant output:

```sql
-- Extra CUSTOMERS (adds 12 more; unique emails to satisfy UNIQUE key)

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Drew','Lopez','drew.l@example.com','555-4040');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Emi','Patel','emi.p@example.com','555-5050');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Fatima','Nguyen','fatima.n@example.com','555-6060');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Gabe','Santos','gabe.s@example.com','555-7070');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Hana','Kim','hana.k@example.com','555-8080');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Ivan','Silva','ivan.s@example.com','555-9090');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Jia','Zhou','jia.z@example.com','555-1112');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Kai','Olsen','kai.o@example.com','555-1113');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Lina','Haddad','lina.h@example.com','555-1114');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Maya','Garcia','maya.g@example.com','555-1115');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Noah','Reed','noah.r@example.com','555-1116');

INSERT INTO customers (first_name,last_name,email,phone) VALUES
('Omar','Khan','omar.k@example.com','555-1117');

-- Extra PRODUCTS (adds 6 more; keep prices clean)

INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-500','Bluetooth Headset','Accessories',59.00);

INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-600','Webcam 1080p','Accessories',39.99);

INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-700','15in Laptop Stand','Peripherals',49.50);

INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-800','Portable SSD 1TB','Storage',119.00);

INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-900','Wireless Charger','Accessories',29.00);

INSERT INTO products (sku,product_name,category,unit_price) VALUES
('SKU-950','Ergonomic Chair','Furniture',219.00);

-- One more STORE (adds a 3rd store)

INSERT INTO stores (store_name,region,city,state_province)
```

```sql
VALUES ('Airport Kiosk','East','Tampa','FL');

-- INVENTORY for all missing (store, product) pairs (keeps totals modest)

-- This adds rows only where you don't already have inventory.

INSERT INTO inventory (store_id, product_id, on_hand_qty, safety_stock_qty,
reorder_point_qty)

SELECT s.store_id,

      p.product_id,

      CASE

        WHEN p.category = 'Storage' THEN 30

        WHEN p.category = 'Furniture' THEN 15

        WHEN p.category = 'Peripherals' THEN 80

        ELSE 120

      END AS start_qty,

      10, 20

FROM stores s

CROSS JOIN products p

WHERE (s.store_id, p.product_id) NOT IN (

 SELECT store_id, product_id FROM inventory

);

-- ORDERS (8 new orders; Order #1 already exists from your earlier test)

-- Order #2: Emi at Northside Mall

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='emi.p@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Northside Mall'),

       'DEBIT','PLACED');



INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (2, (SELECT product_id FROM products WHERE sku='SKU-500'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-500'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (2, (SELECT product_id FROM products WHERE sku='SKU-100'), 3, (SELECT
unit_price FROM products WHERE sku='SKU-100'));

UPDATE orders SET order_status='PAID' WHERE order_id=2;
```

```sql
-- Order #3: Fatima at Downtown Flagship

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='fatima.n@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Downtown Flagship'),

       'CREDIT','PLACED');


INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (3, (SELECT product_id FROM products WHERE sku='SKU-800'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-800'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (3, (SELECT product_id FROM products WHERE sku='SKU-700'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-700'));

UPDATE orders SET order_status='PAID' WHERE order_id=3;

-- Order #4: Gabe at Airport Kiosk

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='gabe.s@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Airport Kiosk'),

       'CASH','PLACED');


INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (4, (SELECT product_id FROM products WHERE sku='SKU-600'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-600'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (4, (SELECT product_id FROM products WHERE sku='SKU-900'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-900'));

UPDATE orders SET order_status='PAID' WHERE order_id=4;


-- Order #5: Hana at Downtown Flagship

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='hana.k@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Downtown Flagship'),

       'CREDIT','PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (5, (SELECT product_id FROM products WHERE sku='SKU-950'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-950'));
```

```sql
INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (5, (SELECT product_id FROM products WHERE sku='SKU-400'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-400'));

UPDATE orders SET order_status='PAID' WHERE order_id=5;

-- Order #6: Ivan at Northside Mall

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='ivan.s@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Northside Mall'),

       'DEBIT','PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (6, (SELECT product_id FROM products WHERE sku='SKU-200'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-200'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (6, (SELECT product_id FROM products WHERE sku='SKU-300'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-300'));

UPDATE orders SET order_status='PAID' WHERE order_id=6;

-- Order #7: Jia at Airport Kiosk

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='jia.z@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Airport Kiosk'),

       'CREDIT','PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (7, (SELECT product_id FROM products WHERE sku='SKU-500'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-500'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (7, (SELECT product_id FROM products WHERE sku='SKU-700'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-700'));

UPDATE orders SET order_status='PAID' WHERE order_id=7;

-- Order #8: Kai at Downtown Flagship

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='kai.o@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Downtown Flagship'),

       'DEBIT','PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)
```

```sql
VALUES (8, (SELECT product_id FROM products WHERE sku='SKU-900'), 3, (SELECT
unit_price FROM products WHERE sku='SKU-900'));

UPDATE orders SET order_status='PAID' WHERE order_id=8;

-- Order #9: Lina at Northside Mall

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='lina.h@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Northside Mall'),

       'CREDIT','PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (9, (SELECT product_id FROM products WHERE sku='SKU-100'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-100'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (9, (SELECT product_id FROM products WHERE sku='SKU-600'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-600'));

UPDATE orders SET order_status='PAID' WHERE order_id=9;

-- Order #10: Maya at Airport Kiosk

INSERT INTO orders (customer_id, store_id, payment_method, order_status)

VALUES ((SELECT customer_id FROM customers WHERE email='maya.g@example.com'),

       (SELECT store_id FROM stores WHERE store_name='Airport Kiosk'),

       'CASH','PLACED');

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (10, (SELECT product_id FROM products WHERE sku='SKU-800'), 1, (SELECT
unit_price FROM products WHERE sku='SKU-800'));

INSERT INTO order_items (order_id, product_id, quantity, unit_price_at_sale)

VALUES (10, (SELECT product_id FROM products WHERE sku='SKU-400'), 2, (SELECT
unit_price FROM products WHERE sku='SKU-400'));

UPDATE orders SET order_status='PAID' WHERE order_id=10;

-- Counts to show you're in the 50–100 range overall

SELECT 'CUSTOMERS' tbl, COUNT(*) cnt FROM customers UNION ALL

SELECT 'STORES',    COUNT(*) FROM stores UNION ALL

SELECT 'PRODUCTS',  COUNT(*) FROM products UNION ALL

SELECT 'INVENTORY', COUNT(*) FROM inventory UNION ALL

SELECT 'ORDERS',    COUNT(*) FROM orders UNION ALL

SELECT 'ORDER_ITEMS', COUNT(*) FROM order_items;

-- Spot-check one order total
```

```sql
SELECT order_id, order_total_amt FROM orders WHERE order_id IN
(1,2,3,4,5,6,7,8,9,10) ORDER BY order_id;

-- Low stock example (handy for Person 4 later)

SELECT s.store_name, p.sku, i.on_hand_qty, i.reorder_point_qty

FROM inventory i

JOIN products p ON p.product_id = i.product_id

JOIN stores   s ON s.store_id   = i.store_id

WHERE i.on_hand_qty <= i.reorder_point_qty

ORDER BY s.store_name, p.sku;
```

- JSON sample documents

```sql
-- Create a demo JSON table
CREATE TABLE order_json_cache (
    order_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_doc   JSON
);

-- Insert a few JSON rows
INSERT INTO order_json_cache (order_doc)
VALUES (JSON('{
  "customer": {"email": "alex.r@example.com"},
  "total_amount": 457.00,
  "items": [
    {"product": "Mechanical Keyboard", "category": "Accessories"},
    {"product": "27in 4K Monitor", "category": "Displays"}
  ]
}'));

INSERT INTO order_json_cache (order_doc)
VALUES (JSON('{
  "customer": {"email": "hana.k@example.com"},
  "total_amount": 243.00,
  "items": [
    {"product": "Ergonomic Chair", "category": "Furniture"},
    {"product": "USB-C Cable 2m", "category": "Cables"}
  ]
}'));

COMMIT;
CREATE OR REPLACE VIEW customer_dv AS
SELECT JSON_OBJECT(
         'customer_id'  VALUE c.customer_id,
         'first_name'   VALUE c.first_name,
         'last_name'    VALUE c.last_name,
         'email'        VALUE c.email,
         'orders'       VALUE (
           SELECT JSON_ARRAYAGG(
                    JSON_OBJECT(
```

```
                      'order_id'        VALUE o.order_id,
                      'order_datetime'  VALUE TO_CHAR(o.order_datetime,
'YYYY-MM-DD"T"HH24:MI:SS'),
                      'order_total_amt' VALUE o.order_total_amt
                    )
                  )
            FROM orders o
            WHERE o.customer_id = c.customer_id
          )
        ) AS dv
FROM    customers c;
SELECT view_name FROM user_views WHERE view_name = 'CUSTOMER_DV';

SELECT column_name, data_type
FROM    user_tab_columns
WHERE   table_name = 'CUSTOMER_DV'
ORDER   BY column_id;
SELECT customer_id FROM customers FETCH FIRST 5 ROWS ONLY;
SELECT JSON_SERIALIZE(dv RETURNING CLOB PRETTY) AS customer_360_json
FROM    customer_dv
WHERE   JSON_VALUE(dv, '$.customer_id') = 1;  -- change 1 to an id that exists

CREATE OR REPLACE VIEW order_dv AS
SELECT JSON_OBJECT(
          'order_id'       VALUE o.order_id,
          'customer_id'    VALUE o.customer_id,
          'store_id'       VALUE o.store_id,
          'order_datetime' VALUE TO_CHAR(o.order_datetime,
'YYYY-MM-DD"T"HH24:MI:SS'),
          'order_total'    VALUE o.order_total_amt,
          'items'          VALUE (
            SELECT JSON_ARRAYAGG(
                     JSON_OBJECT(
                       'product_id' VALUE oi.product_id,
                       'quantity'   VALUE oi.quantity,
                       'unit_price' VALUE oi.unit_price_at_sale,
                       'line_total' VALUE oi.line_amount
                     )
                   )
            FROM order_items oi
            WHERE oi.order_id = o.order_id
          )
        ) AS dv
FROM    orders o;

SELECT order_id FROM orders FETCH FIRST 5 ROWS ONLY;

SELECT JSON_SERIALIZE(dv RETURNING CLOB PRETTY) AS order_json
FROM    order_dv
WHERE   JSON_VALUE(dv, '$.order_id') = 1;   -- replace 1 with a real order id
```

- Screenshots of dashboards or query results

```
SELECT o.order_id,
       JSON_VALUE(o.order_doc, '$.customer.email') AS customer_email,
       JSON_VALUE(o.order_doc, '$.total_amount')    AS order_total
FROM   order_json_cache o
WHERE  JSON_EXISTS(o.order_doc, '$.items[*]?(@.category == "Accessories")')
ORDER  BY o.order_id;
```

Relational Modeling (OLTP)

	ORDER_ID	ORDER_TOTAL_AMT
1	1	457

That "457" means our transactional layer is correctly calculating and syncing totals (it's the clearest sign that your OLTP foundation is solid.)

	STORE_ID	SKU	ON_HAND_QTY
1	1	SKU-200	48
2	1	SKU-300	19

Inventory decreased after sales

- Any additional diagrams or artifacts

6. References

- Any external resources, documentation, literature or tutorials used to support the project (APA 7th edition style guide for all formatting)